PROJECT PROPOSAL · CONFIDENTIAL

Release Sentinel

AI-Powered Release Risk Governance System

Preventing production incidents before they ship — using AI to evaluate every release candidate against the intelligence of every incident your team has ever experienced.

40–60%	4–6	3 / 4	50%
Fewer incidents	P1s prevented / year	Past incidents were AI-preventable	MTTR reduction via auto root-cause

Prepared for: VP Engineering · Head of DevOps · Chief Risk Officer

April 2026

01 · THE PROBLEM

Production releases are high-stakes guesses

Release managers today rely on three signals — test coverage, code review completion, and deployment checklists. None of these are predictive. They tell you what was tested, not what will break in production.

> **The missing intelligence:** Which specific code changes historically cause production failures?

Last 12 months — incidents with detectable patterns

Incident	Root Cause	Cost	AI-Preventable?
NAV-2547	Fee calc edge case — leap year	$180K	✓ Yes — prior pattern existed
REC-8901	Recon break — same module, 3 prior P1s	$220K	✓ Yes — module history flagged
REG-5432	Reg report missing data after schema change	$140K	✓ Yes — coverage gap on migration
PERF-7721	DB deadlock from new query pattern	$95K	■ Partial — load-dependent

3 out of 4 P1 incidents had detectable patterns in historical data before they shipped.

$100K – $250K	80 hrs	3 / 4
Average cost per P1 SLA + remediation + trust	Engineering hours / incident 6–8 team members	Incidents preventable by AI Based on last 12 months

02 · THE SOLUTION

Release Sentinel — AI that learns from every incident

Release Sentinel is a RAG-powered (Retrieval-Augmented Generation) system that analyses every production release candidate automatically and answers one question:

"Does this code look like something that has broken production before?"

How it works — end to end

1 Trigger

Developer merges a PR to the release branch in Bitbucket. Webhook fires automatically — no manual step required.

2 Resolve release manifest from JIRA

All stories and epics in the release are fetched. Linked PRs are resolved. The full code change surface is aggregated.

3 Multi-agent evaluation (5 parallel AI agents)

Five specialist agents evaluate the code diff simultaneously — code debt, performance, design patterns, security, and RAG incident similarity. Each runs in isolation with its own focused instruction set from S3.

4 Risk score and evidence report

A weighted aggregator combines all five verdicts into a score (0–100) with specific named evidence — incident references, PR numbers, similarity scores. Posted as a Bitbucket PR comment within 30 seconds.

5 Governance gate

Score 0–70: auto-proceed. Score 70–85: senior approval required via MS Teams. Score 85+: deployment blocked. Full audit trail retained for 7 years.

Why RAG over a trained ML model

Capability	Traditional ML	Release Sentinel RAG
Time to value	3–4 months (labelling + training)	Accelerated — Blink provides production-grade vector store, LLM gateway, and job queue
Data requirements	6–12 months of labelled deployments	Works from day one with existing incidents
Explainability	Abstract feature importance scores	Exact named incident + similarity score
Maintenance	Monthly retraining cycle	Auto-updates as incidents are ingested
Cold start	Poor until 100s of labels	Intelligent from day one

03 · WORKFLOW

How Release Sentinel works — end to end

The diagram below shows the complete lifecycle from trigger to post-deployment monitoring. Every release candidate is scored, gated, and audited automatically — with the knowledge base growing with every incident your team encounters.

Fig 1 — End-to-end flow: Trigger → RAG engine → Governance gate → Post-deployment monitoring

Three trigger points, one analysis engine

Trigger	When it fires	Mode	Can block deploy?
Bitbucket PR merged	Developer merges to release branch	Async — PR comment in 30s	No — informational
Release manager submission	Release manager submits JIRA fix version	Async — full release report	No — informational
Jenkins pre-deploy gate	Automatically at pre-deploy pipeline stage	Sync — blocks pipeline	Yes — hard gate

Post-deployment: Release Sentinel monitors production for 48 hours after every release. Anomalies are correlated to the specific PR that introduced them. Rollback runbooks are auto-generated and attached to the incident record.

04 · TECHNICAL ARCHITECTURE

Leveraging Blink as a proven AI infrastructure foundation

Release Sentinel is a purpose-built, substantive system with significant implementation depth — spanning a multi-agent evaluation engine, a dual-embedding RAG pipeline, an S3 knowledge base with intelligent document parsing, governance workflow automation, and a full audit trail. Crucially, it is designed to leverage Blink's existing production-grade AI infrastructure — the vector store, LLM gateway, embedding services, async job framework, and auth layer — eliminating the highest-risk parts of the build and accelerating delivery without compromising on capability.

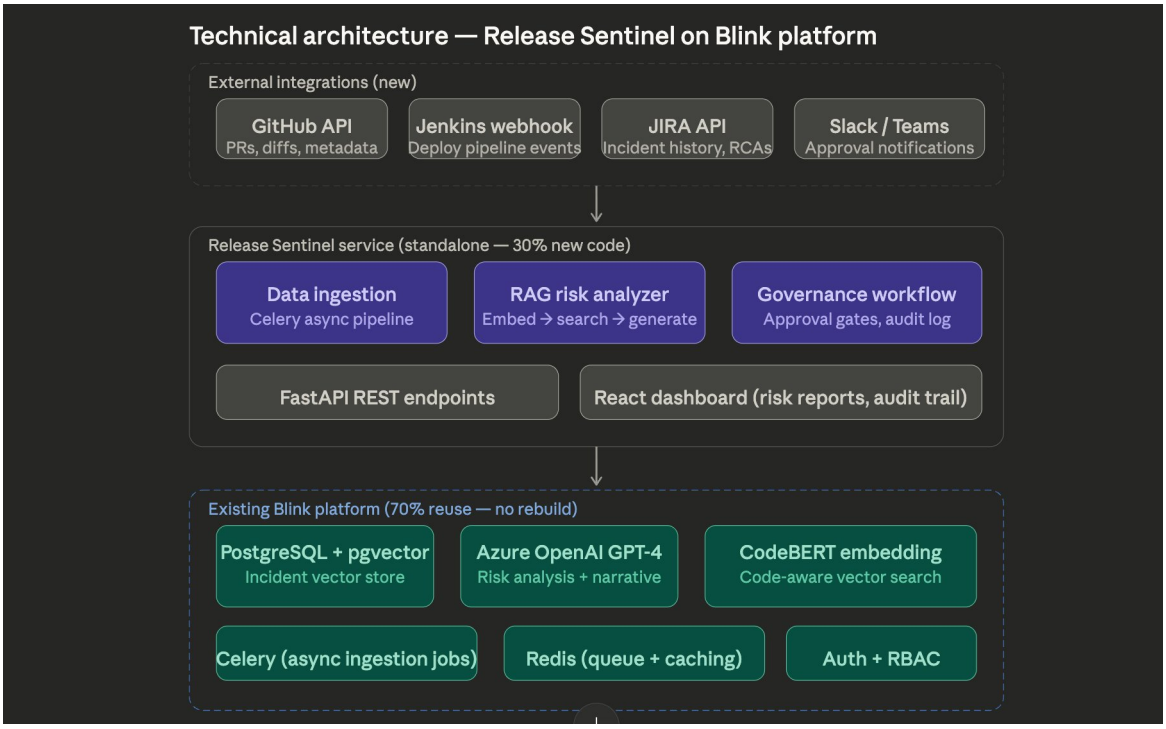

Fig 2 — Technical architecture: external integrations → Release Sentinel → Blink foundation

External integrations

- Bitbucket webhooks (PR merged → analysis trigger)
- JIRA REST API (release manifest — stories + linked PRs)
- Jenkins pre-deploy webhook (governance gate)
- MS Teams (risk reports + senior approval workflow)

Release Sentinel — core implementation

- Multi-agent evaluation engine — 5 parallel specialist agents with S3-backed instruction store
- RAG pipeline — dual embedding (CodeBERT + text-embedding-3-large), pgvector ANN search, recency decay
- S3 knowledge base — intelligent document ingestion, phase-aware chunking, context header embedding
- Governance engine — approval gates, hard override rules, 7-year audit trail
- FastAPI REST endpoints + React risk dashboard

Blink platform — leveraged as the AI infrastructure foundation

- ◆ PostgreSQL + pgvector — production vector store, HNSW indexes, hybrid retrieval queries
- ◆ Azure OpenAI GPT-4 — LLM gateway with existing quota, rate limiting, and failover
- ◆ CodeBERT embedding service — code-aware vector search, GPU node already provisioned in AKS
- ◆ Celery + Redis — battle-tested async job queue for parallel agent dispatch
- ◆ FastAPI framework · Auth + RBAC — reused across all Release Sentinel endpoints

05 · MULTI-AGENT DESIGN

Five specialist agents — parallel evaluation

A single LLM prompt checking for everything — debt, performance, security, patterns, and incident similarity — suffers from context bloat, diluted focus, and hallucination drift. Five specialist agents in isolation, each with a narrow focused persona, solve all three problems.

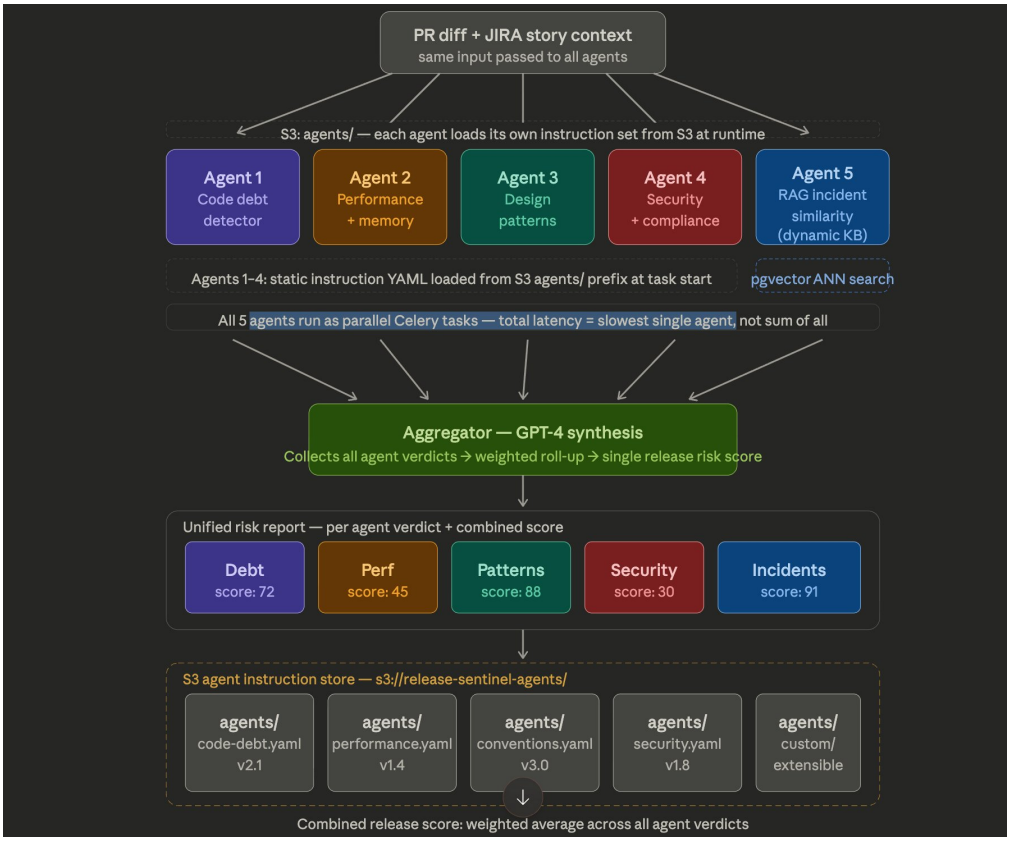

Fig 3 — Multi-agent pipeline: same PR diff evaluated in parallel by five specialist agents

Same PR diff → all five agents in parallel

Agent 1	Agent 2	Agent 3	Agent 4	Agent 5
Code Debt	Performance	Design Patterns	Security	RAG Incident Similarity

All 5 agents run as parallel Celery tasks — total latency = slowest single agent (~8s), not sum of all (~40s)

Agent	Evaluates	Instruction source	Weight
1 — Code debt	TODOs, duplication, missing abstractions, bypassed utilities	S3: code-debt.yaml	15%
2 — Performance	N+1 queries, unbounded loops, memory leaks, blocking I/O	S3: performance.yaml	20%
3 — Design patterns	Layering violations, naming deviations, anti-patterns	S3: conventions.yaml	15%

4 — Security	Hardcoded secrets, SQL injection, missing auth, PII exposure	S3: security.yaml	25%
5 — RAG incidents	Similarity to past code patterns that caused production P1s	Dynamic — pgvector ANN	25%

Hard override rule: If the Security or RAG Incident agent returns a FAIL verdict, the aggregated release score is floored at 75 — a critical finding cannot be averaged away by clean scores from other dimensions.

06 · KNOWLEDGE BASE

S3 incident KB + dual embedding strategy

The RAG knowledge base accepts whatever artefacts your team already produces during and after incidents — postmortems, runbooks, Slack thread exports, emails, ad hoc notes. Format-agnostic: drop a file in the S3 bucket and the ingestion pipeline handles everything automatically. No structured forms, no JIRA templates, no manual indexing.

S3 incident knowledge base

s3://release-sentinel-incident-kb/

- postmortems/ — Markdown, PDF, Confluence exports
- runbooks/ — Markdown, YAML, Word documents
- slack-exports/ — JSON incident thread exports
- adhoc/ — Emails, notes, screenshots

Drop a file → S3 event fires → Celery ingests → embedded and searchable in under 5 minutes.

Bootstrapping with existing artefacts

Export Confluence incident spaces, Git-tracked runbooks, and Slack incident channels. The knowledge base is operational on day one.

S3 agent instruction store

Agent instruction YAMLs are versioned in S3. Tech leads can update evaluation criteria without a code deploy — changes take effect on the next PR merge. Git-controlled with mandatory review.

Dual embedding strategy

Two models — one for code, one for prose — because they live in completely different semantic spaces.

Model	Used for	Dims
CodeBERT	PR code diffs	768
text-embedding-3-large	Incident documents	3072

Why CodeBERT? A general model sees auto_resolve(txn) and process_breaks(items) as different functions. CodeBERT sees the same dangerous loop-without-null-guard pattern — similarity 0.91. That is the signal that catches bugs before they ship.

Both searches run in parallel (asyncio.gather)

- CodeBERT → searches code_embeddings for known-bad PR patterns
- text-embedding-3-large → searches incident_chunks for postmortems
- Recency decay — incidents from 6 months ago weighted at 50%
- GPT-4 receives both result sets → unified evidence report

07 · GOVERNANCE & RISK

Auditable gates — proactive risk management

Governance gate design

0 – 70	70 – 85	85 – 100
Auto-proceed Risk report posted to PR + MS Teams	Senior approval required MS Teams approval card sent · Jenkins pauses	Auto-deploy blocked Jenkins halted · on-call alerted

Risk management

Risk	Likelihood	Mitigation
False positives blocking valid releases	Medium	Shadow mode first — scores computed without enforcement. Calibrate thresholds against historical data. Target < 15% false positive rate before enforcement goes live.
Knowledge base too small at launch	Low	RAG works from any corpus size. Bootstrapping plan (Confluence, Slack exports, Git runbooks) ensures meaningful signal before go-live.
GPT-4 / Azure OpenAI unavailability	Low	Fail-open policy — if analysis unavailable, deployment proceeds with a Teams warning. Never blocked by infrastructure failure.
Agent instructions drifting quality	Medium	Git-controlled YAML with mandatory tech lead review. Retrospective validation runs against historical PRs before every instruction change is deployed.
Developer resistance to the gate	Low	Shadow mode first. Evidence-based reports (named incidents, specific line numbers) are convincing — developers engage with specific findings, not abstract scores.

08 · DELIVERY APPROACH

Phased rollout — shadow mode before enforcement

The delivery is structured in four phases, each with a clear success criterion before proceeding to the next. Gate enforcement is the last step — not the first.

Phase	Focus	Key deliverables	Gate to proceed
Phase 1 Foundation	Data + RAG core	S3 buckets · pgvector schema · document ingestion pipeline · CodeBERT GPU deployment · MVP risk scoring API	Retrospective score > 70 for known past incidents
Phase 2 Governance	Multi-agent + gates	All 5 agents · JIRA + Bitbucket integration · Governance gate logic · MS Teams approval workflow · Audit trail	Full release → risk report in under 60 seconds
Phase 3 UI + Integrations	Dashboard + Jenkins	React dashboard · Jenkins pre-deploy gate · Post-deployment 48h monitoring · Rollback runbook generation	Full integration test on real release candidate
Phase 4 Pilot → Production	Shadow mode → live	Shadow mode pilot (scores computed, gates not enforced) · Threshold calibration · False positive analysis · Gate enforcement	Under 15% false positive rate at the 70-score threshold

09 · NEXT STEPS

What we need to proceed

1 Stakeholder alignment (today)

Agreement on the proposal direction. Identification of the right team members. Access alignment to Blink platform, Bitbucket, JIRA, and Jenkins environments.

2 Phase 1 scoping

Define the initial incident corpus for bootstrapping. Agree on release branches and JIRA projects in scope. Confirm Azure OpenAI quota and pgvector access on the Blink platform.

3 Shadow mode pilot definition

Agree which release team runs the pilot in Phase 4. Define success metrics — false positive rate target, latency SLA, coverage.

4 Agent instruction authoring

Tech leads draft the initial YAML instruction files for all four static agents — code debt, performance, conventions, and security — based on your existing review checklists.

Release Sentinel transforms release risk from a guess into a data-driven decision. Every incident your team experiences makes the next release safer.